StepWise: An Explainable Classical Machine Learning Tutor for Diagnosing Misconceptions in Handwritten Mathematics

Siddhi J Salunkhe, Sanvi Saket, and Prof. Manjula R

Department of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India

Abstract. Students make systematic, repeatable mistakes in arithmetic and early algebra, but teachers rarely have time to give individual feedback on handwritten work. Existing tools either solve the problem for the student or rely on large neural and language models that are opaque, costly and can state mathematically wrong explanations. We present StepWise, an explainable tutoring system that reads a photograph of handwritten working, proves which step is wrong, names the likely misconception and recommends targeted practice, using only classical machine learning. Handwritten symbols are segmented with OpenCV and classified by a support vector machine on histogram-of-oriented-gradient features; every step is verified exactly with SymPy; a random forest over 33 interpretable features classifies the misconception; a second classifier trained on 1,720 real student mistakes separates careless slips from understanding gaps; and Bayesian knowledge tracing with item response theory drives adaptive practice. On the official CROHME 2014/2016/2019 test sets (unseen writers) the recognizer reaches 92.4% symbol accuracy (95% CI 91.2–93.6%), against 55.4% when trained on fonts only, and reads whole expressions twice as often as a zero-shot transformer OCR baseline (56.7% vs. 28.4% exact match, $p < 10^{-6}$) while being 28 times faster and 160 times smaller. An illumination-normalisation step raises accuracy under uneven lighting from 3.1% to 56.7%. On ScratchMath, a classifier trained on 1,720 real student mistakes separates careless slips from understanding gaps with 68.3% accuracy against a 55.2% majority baseline ($p < 0.001$), and on ASSISTments 2009 (446,844 real answers) knowledge tracing predicts the next answer with an AUC of 0.815. The full system runs offline on a laptop CPU.

Keywords: Intelligent tutoring · Handwritten mathematics · Misconception diagnosis · Support vector machine · Knowledge tracing · Explainable AI.

1 Introduction

Arithmetic and early algebra errors are rarely random. A student who writes $-3 + 5 = -8$ or moves $+4$ across the equals sign without changing its sign is usually following a consistent but faulty procedure, a "bug" in the sense of Brown

and Burton [1]. Diagnosing these bugs early is valuable because each one can be repaired with a short, targeted explanation. In practice, however, a teacher marking a class set of handwritten worksheets can only mark answers right or wrong; the information about *why* an answer is wrong stays on the page.

Software can help, but current tools address a different problem. Photo-solver applications recognise a printed problem and return the solution, which removes the need for the student to think. Intelligent tutoring systems such as Cognitive Tutor [2] diagnose errors well but require typed, step-by-step input in a dedicated interface. Recent approaches based on deep networks and large language models can read handwriting and generate explanations, but they need GPUs or paid APIs, are hard to audit, and can confidently state incorrect mathematics [3].

This paper presents StepWise, a tutoring system built around three design decisions. First, *exact mathematics is used wherever truth can be proved*: every line of the student's working is verified symbolically, so the system never invents an error. Second, *classical machine learning is used wherever there is genuine uncertainty*: reading handwriting, deciding which kind of mistake was made, estimating mastery and grouping students. Third, *every learned component is evaluated on real, publicly available data* with baselines and held-out test sets. The main contributions are:

- an end-to-end pipeline from a photograph of handwritten working to a verified, explained misconception diagnosis and adaptive practice, running offline on a CPU;
- a learned "bug library": classical misconception rules turned into 33 named features for a random forest, so that each diagnosis is accompanied by the evidence that produced it;
- a count-alignment procedure that harvests labelled real handwritten symbols from CROHME expression-level annotations, and an evaluation showing that training on real handwriting raises symbol accuracy on unseen writers from 55.4% to 92.4%;
- a head-to-head comparison with a transformer OCR model on the same real test expressions, and a robustness study under simulated phone-photo conditions with an illumination-normalisation step;
- an evaluation of knowledge tracing on ASSISTments 2009 and of a slip-versus-understanding classifier on ScratchMath, the only public dataset of real handwritten student mistakes with error labels that we are aware of, with bootstrap confidence intervals, significance tests and feature ablations.

2 Related Works

Diagnosing procedural errors. Brown and Burton [1] modelled students' subtraction errors as "bugs" in otherwise correct procedures and built a library of such bugs to explain wrong answers. VanLehn [4] extended this into a theory of how bugs arise. Model-tracing tutors [2] follow each typed step and match it against correct and buggy production rules. StepWise keeps the idea of a bug

Table 1. Datasets used in this work.

Dataset	Type	Size used	Purpose
CROHME train [5]	real	6,271 symbols from 1,337 expressions	symbol recognizer training
CROHME 2014/16/19 test [6]	real	1,821 symbols; 127 expressions	symbol and expression testing (unseen writers)
HASYv2 [12]	real	3,335 symbols, 144 writers	symbol recognizer training
ASSISTments 2009 [18]	real	446,844 answers, 4,151 students, 110 skills	knowledge tracing
ScratchMath [3]	real	1,720 wrong answers with scratchwork photos	error-type classifier
Font-rendered glyphs	synthetic	12,210 glyphs from 24 fonts	robustness to printed input
Generated solutions	synthetic	3,150 solutions, 9 classes	fine-grained misconception classifier

library but encodes each bug as a feature and lets a classifier learn how to weigh overlapping evidence, instead of relying on hand-ordered rules.

Handwritten mathematical expression recognition. The CROHME competitions [5,6] established the standard benchmark for handwritten mathematical expressions. Recent systems are encoder–decoder neural networks such as WAP [7] and transformer-based OCR such as TrOCR [8]. These achieve high accuracy on full two-dimensional expressions but require GPU training and large models. For isolated characters, histogram-of-oriented-gradient (HOG) features [9] with support vector machines [10] remain a strong and efficient baseline [11]; the HASYv2 dataset [12] provides real handwritten symbols from many writers.

Knowledge tracing. Bayesian Knowledge Tracing (BKT) [13] models each skill as a hidden learned/unlearned state. Performance Factors Analysis (PFA) [14] instead uses logistic regression on counts of past successes and failures. Deep Knowledge Tracing [15] uses recurrent networks, although later work showed that well-specified classical models remain competitive [16]. Item response theory [17] estimates item difficulty and learner ability. ASSISTments [18] provides the most widely used public benchmark.

Datasets of student mistakes. Public data on *why* students err is scarce. ScratchMath [3] provides real handwritten scratchwork from primary and middle-school students with expert error categories; its authors show that even multimodal large language models struggle to identify the error type. StepWise is, to the best of our knowledge, the first system to combine symbolic step verification, classical misconception classification and knowledge tracing in one offline pipeline evaluated on these real datasets.

3 Dataset

Table 1 lists the datasets. All real datasets are downloaded from their public sources by a script and are never redistributed with the code.

Harvesting real symbols from CROHME. CROHME annotates whole expressions in LaTeX but provides no symbol bounding boxes in the image release

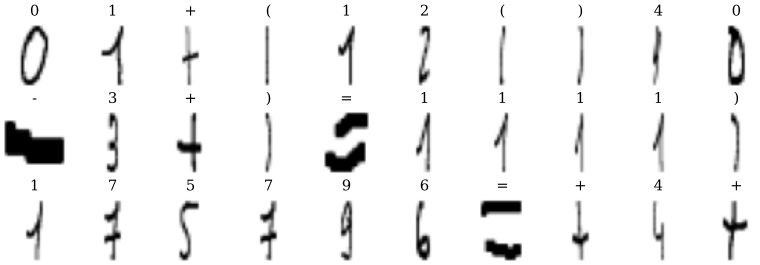


Fig. 1. A random sample of real handwritten symbols harvested from CROHME by count alignment, after size normalisation (labels above). Thick, blocky strokes are artifacts of the stretched CROHME renderings.

we use. We keep only *linear* expressions (no fractions, scripts or roots), segment each with the same OpenCV segmenter used at inference time, and, when the number of segmented components equals the number of LaTeX tokens, label the crops in left-to-right order. Expressions where the counts differ are discarded. Of 2,185 linear training expressions, 1,337 aligned, giving 6,271 labelled symbols over 17 classes (`0`–`9`, `+ - = x ( ) /`, with $\times$ mapped to `x`). A manual check of 80 random crops found 79 correctly labelled. Fig. 1 shows examples.

Real student mistakes. ScratchMath contains 1,720 wrong answers from Chinese primary (1,479) and middle-school (241) students. Each record holds the question, the correct answer, a reference solution, the student's answer, a photograph of the student's handwritten scratchwork and one of seven expert error categories: calculation (566), question misunderstanding (519), concept (219), strategy (135), copying (124), reasoning (75) and attention (82). The expert's free-text explanation is never used as an input, because it reveals the label.

4 Methodology

4.1 System Overview

Fig. 2 shows the pipeline. A photograph is cleaned and segmented, each symbol is classified, the recognised lines are verified symbolically, and the verified steps feed a misconception classifier. Its output drives feedback, mastery tracking, practice selection and the teacher's class view. Symbols the recognizer is unsure about are highlighted; when a student corrects one, the crop is stored as labelled training data for the next model version.

4.2 Preprocessing and Segmentation

The image is corrected for EXIF orientation and converted to greyscale. Photographs of paper are often unevenly lit, which breaks a global threshold, so the

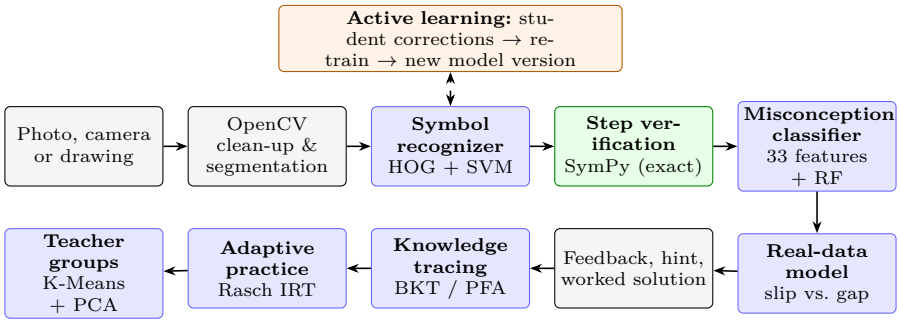


Fig. 2. StepWise architecture. Blue boxes are learned components; the green box is exact symbolic computation.

background is estimated and divided out before binarisation:

$$B = G_\sigma * \big(I \bullet E_k\big), \qquad I_{\text{norm}} = \min\!\Big(255,\ 255\,\frac{I}{B}\Big), \tag{1}$$

where $I \bullet E_k$ is a grey-level morphological closing with an elliptical element of size k (about one twelfth of the shorter image side), which removes thin dark strokes and leaves the background, and G_σ is a Gaussian blur. I_{norm} is then binarised with Otsu's threshold. Text lines are found from the horizontal projection profile, and symbols within a line are the connected components of ink. Components whose horizontal extents overlap by at least half of the narrower one (the two bars of "=", a broken digit) are merged into one symbol. Spacing in the output is canonical rather than taken from the handwriting, and an `x` written between two numbers is interpreted as multiplication.

4.3 Symbol Recognition

Each symbol crop is scaled, preserving its aspect ratio, into a 28×28 canvas. For each pixel, the gradient magnitude and orientation are

$$m(u,v) = \sqrt{G_x(u,v)^2 + G_y(u,v)^2}, \qquad \theta(u,v) = \operatorname{atan2}\big(G_y(u,v), G_x(u,v)\big), \tag{2}$$

and HOG features [9] accumulate m into 9 orientation bins over 7×7-pixel cells grouped into overlapping blocks (324 values). These are concatenated with a 7×7 grid of ink densities (49 values) and the log aspect ratio of the bounding box (1 value), giving a 374-dimensional vector $\mathbf{x}$. A one-versus-one support vector machine [10] with the radial basis kernel classifies the symbol:

$$f(\mathbf{x}) = \operatorname{sign}\Big(\sum_i \alpha_i y_i\, K(\mathbf{x}_i, \mathbf{x}) + b\Big), \qquad K(\mathbf{x}, \mathbf{x}') = \exp\big(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2\big). \tag{3}$$

Platt-scaled probabilities give a confidence for each symbol and its runner-up alternatives, which the interface uses to highlight symbols for the student to

confirm. The SVM was compared with random forest, k-nearest neighbours and logistic regression (Section 5.1).

4.4 Symbolic Step Verification

Each recognised line is parsed with SymPy [19] under a strict character whitelist and a restricted namespace; Python's `eval` is never used. A purely numeric line is true if lhs − rhs simplifies to zero. For equations in x, line k is reduced to its solution set S_k, and the first incorrect step is

$$k^* = \min\{\, k > 1 : S_k \neq S_{k-1} \,\}, \tag{4}$$

so a step is judged by whether it preserves the solution, not by whether it matches a model answer.

4.5 Misconception Classification

For the first incorrect step, 33 named features describe *how* it is wrong. Examples are: whether the written value equals the correct value with one term's sign flipped; whether it matches a different operator; whether it is off by one, two or a power of ten; and, for an equation $ax+b=c$ followed by an incorrect line with solution v, indicators of common algebra bugs:

$$\phi_{\text{transpose}} = \mathbf{1}\big[v = \tfrac{c+b}{a}\big], \qquad \phi_{\text{no-divide}} = \mathbf{1}\big[v = c-b\big], \qquad \phi_{\text{negate}} = \mathbf{1}\big[v = -\tfrac{c-b}{a}\big]. \tag{5}$$

Further features describe OCR confidence, the number of steps, whether work was shown and whether the final answer matches the key. A random forest [20] of 300 trees averages per-tree class probabilities, $p(y \mid \mathbf{x}) = \frac{1}{T}\sum_{t=1}^{T} p_t(y \mid \mathbf{x})$, over nine categories (correct, arithmetic slip, sign error, wrong operation, moving terms incorrectly, skipped step, final-answer mismatch, unreadable, insufficient information). The features that fire are shown to the student as the reasons for the diagnosis. Because no public dataset labels real worked solutions with these fine-grained categories, this model is trained on 3,150 generated solutions, each containing one injected misconception; its results are therefore an upper bound.

Real-data error-type model. To ground the diagnosis in real behaviour, a second classifier is trained on ScratchMath. Its 42 language-independent features describe the relation between the wrong and correct answer (sign flip, decimal-point shift, one-digit difference, digit transposition, an intermediate value of the reference solution, a missing multiplication or division by a number in the question, swapped answer blanks), simple statistics of the question, and statistics of the scratchwork image (ink fraction, stroke count, number of written lines). It predicts both the seven categories and a binary label that groups calculation, copying and attention errors as *slips* and the rest as *understanding gaps*. The distinction matters pedagogically: slips call for checking habits, gaps for re-teaching.

4.6 Knowledge Tracing and Adaptive Practice

Each submission and practice answer is mapped to one of eight skills. BKT [13] models the probability $P(L_t)$ that a skill is learned. With guess G, slip S and learning rate T, the predicted probability of a correct answer and the update after observing a correct answer are

$$P(\text{correct}_t) = P(L_t)(1-S) + \big(1-P(L_t)\big)G, \tag{6}$$

$$P(L_t \mid \text{correct}) = \frac{P(L_t)(1-S)}{P(L_t)(1-S) + \big(1-P(L_t)\big)G}, \tag{7}$$

$$P(L_{t+1}) = P(L_t \mid o_t) + \big(1-P(L_t \mid o_t)\big)\,T, \tag{8}$$

where o_t is the observed answer (the update for a wrong answer is analogous), with parameters fitted per skill by expectation–maximisation. PFA [14] predicts the next answer with logistic regression,

$$\operatorname{logit} P(\text{correct}) = \beta_k + \gamma_k \log(1+s_k) + \rho_k \log(1+f_k) - \delta\, b_i + \boldsymbol{\lambda}^{\top}\mathbf{r}, \tag{9}$$

where s_k, f_k count past successes and failures on skill k, b_i is the Rasch difficulty of item i, estimated from $P(\text{correct}) = \sigma(\theta_j - b_i)$ [17], and $\mathbf{r}$ contains optional recency features (an exponential moving average of recent answers on the skill and indicators for the last answer). The recommender selects the student's diagnosed or least-mastered skill and offers items whose predicted probability of success is closest to 0.7.

4.7 Teacher View and Active Learning

Each student is represented by the rate of each error category and the mastery of each skill. K-Means, with k chosen by the silhouette score, groups the class, and principal component analysis gives a two-dimensional map; each group is named after its most elevated features. Corrections made by students in the interface are saved as labelled crops; retraining adds them to the training set, compares the new model with the active one on the CROHME test sets, and registers it as a new version that is activated only on request.

5 Results and Discussion

All models were trained with scikit-learn [21] on a laptop CPU with a fixed random seed. Model and training-data choices were made on validation data only; all reported test results are on data never used for fitting or selection.

5.1 Symbol Recognition on Real Handwriting

Validation used 15% of the CROHME training expressions and 25% of the fonts; the selection score was the mean macro-F1 over both, because the system must

Table 2. Symbol recognition on the CROHME 2014/2016/2019 test sets (1,821 symbols, unseen writers). All models trained on the same data (fonts + CROHME + HASYv2, excluding validation).

Algorithm	Val. score	Test acc.	Test macro-F1	Fit (s)
SVM (RBF)	**0.930**	**0.926**	**0.899**	32.8
Random forest	0.916	0.900	0.875	6.7
Logistic regression	0.911	0.903	0.880	4.8
k-nearest neighbours	0.907	0.878	0.848	< 0.1

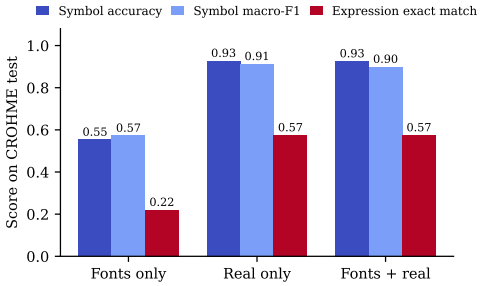


Fig. 3. Effect of training data on real-handwriting performance (same SVM).

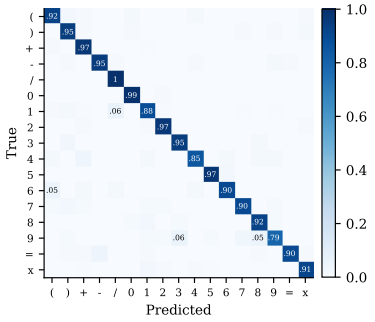


Fig. 4. Row-normalised confusion matrix on the CROHME test sets.

read handwriting and printed worksheets. Testing used the official CROHME 2014, 2016 and 2019 test sets, which were written by people outside the training data. Table 2 compares the algorithms.

The SVM was significantly more accurate than the random forest trained on the same data (McNemar test: 72 symbols correct only for the SVM vs. 26 only for the random forest, $p = 5.5 \times 10^{-6}$). The deployed model, refitted with the validation data included, reached 92.4% accuracy (95% bootstrap CI 91.2–93.6%) and a macro-F1 of 0.898 (0.878–0.915).

Fig. 3 isolates the effect of the training data with the same SVM. A model trained only on font-rendered, augmented glyphs reached a macro-F1 of 0.975 on held-out *fonts*, but only 0.573 on real handwriting: fonts are a poor proxy for writers. Adding real handwriting raised test accuracy from 0.554 to 0.926 and the fraction of whole expressions read exactly from 0.22 to 0.57, and lowered the expression character error rate from 0.452 to 0.110. Training on real data alone gave a slightly higher symbol macro-F1 (0.913) but misread printed digits; the combined model was selected on validation because it handles both. Results were stable across test years (macro-F1 0.891, 0.893 and 0.909 for 2014, 2016 and 2019). The confusion matrix (Fig. 4) shows the remaining errors concentrated on visually similar pairs such as `1`/`/` and `x`/`+`.

Table 3. StepWise vs. a zero-shot transformer OCR on 127 real CROHME test expressions (laptop CPU). 95% bootstrap CIs in brackets.

System	Exact match	CER	ms/expr.	Size
StepWise (HOG + SVM)	**0.567** [0.48, 0.65]	**0.115**	**48**	**8 MB**
TrOCR-base (zero-shot)	0.284 [0.21, 0.36]	0.369	1,325	1,336 MB

5.2 Comparison with a Deep-Learning OCR Baseline

To test whether a lightweight classical model is competitive, we ran TrOCR [8] (`trocr-base-handwritten`, 334M parameters) on the same 127 CROHME test expressions that fall within StepWise's symbol vocabulary. Each expression was cropped to its ink and passed to TrOCR as a text line. Because TrOCR was trained on general English handwriting, its output was normalised leniently before scoring: only digits and the seven operator symbols were kept, $\times$ was mapped to `x`, and spacing was made canonical. Table 3 shows that, without maths-specific fine-tuning, TrOCR read half as many expressions exactly as StepWise (McNemar test on exact matches: 40 vs. 4 expressions correct only for one system, $p = 1.3 \times 10^{-7}$), while needing 28 times more time per expression on the same CPU and about 160 times more storage. TrOCR could be fine-tuned on CROHME, but that requires GPU training; the comparison shows that a classical pipeline trained on real data is a strong baseline for this restricted vocabulary.

5.3 Robustness to Phone-Photo Conditions

CROHME images are clean renderings, whereas students will photograph paper. We therefore degraded every test expression with Gaussian blur ($\sigma = 1.5$), Gaussian noise ($\sigma = 25$), JPEG compression (quality 20), a 5° rotation, a horizontal lighting gradient, and a combination of rotation, lighting gradient, blur, noise and JPEG compression, and measured whole-expression accuracy with and without illumination normalisation (Table 4). Blur, noise and compression had little effect, and rotation cost about eight points because the segmenter assumes horizontal lines. Uneven lighting was catastrophic for global thresholding (3.1% exact) and was fully repaired by normalisation (56.7%, identical to clean images). Under the combined photo-like degradation, normalisation raised exact match from 32.3% to 51.2% and cut the character error rate from 0.608 to 0.119. Clean-image accuracy was unchanged, so normalisation is enabled by default.

5.4 Misconception Diagnosis

On the held-out 20% of the generated solutions, the random forest reached a macro-F1 of 0.994, ahead of logistic regression (0.992), gradient boosting (0.991) and a depth-8 decision tree (0.979). The hand-written rule-based diagnoser that the system previously used reached only 0.330: it had zero recall for wrong

Table 4. Whole-expression accuracy under simulated phone-photo degradations (127 CROHME test expressions), without and with illumination normalisation.

	Exact match		CER	
Condition	without	with	without	with
Clean	0.567	0.567	0.115	0.115
Blur	0.559	0.559	0.114	0.113
Noise	0.575	0.575	0.119	0.119
JPEG (q = 20)	0.575	0.575	0.115	0.115
Rotation 5°	0.488	0.480	0.131	0.133
Uneven lighting	0.031	**0.567**	0.528	**0.115**
Combined photo-like	0.323	**0.512**	0.608	**0.119**

Table 5. Error-type classification on ScratchMath (real student mistakes, held-out test set, $n = 344$).

	Slip vs. understanding	Seven error types	
Model	Accuracy (AUC)	Accuracy	Macro-F1
Majority baseline	0.552 (–)	0.329	0.071
Random (stratified) baseline	0.480 (–)	0.267	0.161
Logistic regression	**0.683** (0.765)	0.340	0.307
Random forest	0.689 (0.772)	**0.491**	**0.359**
Gradient boosting	0.704 (0.772)	0.462	0.352

operations, moving terms incorrectly and skipped steps, and it labelled incorrect algebra as correct whenever no answer key was given. Because the data are synthetic, these scores show that the features separate the modelled error types; they are not estimates of classroom accuracy.

Table 5 reports the classifier trained on real mistakes (stratified 80/20 split, selection by 5-fold cross-validation on the training part). Distinguishing slips from understanding gaps is learnable from answer and scratchwork features: logistic regression reached 68.3% accuracy (95% CI 63.1–73.0%) and an AUC of 0.76 (0.71–0.81), against 55.2% for the majority class; the difference is significant (McNemar test, $p = 9.4 \times 10^{-4}$). The seven-way task is harder because the categories overlap; the random forest reached 49.1% accuracy (macro-F1 0.359) against 32.9% for the majority baseline. Adding character n-grams of the Chinese question text raised the seven-class macro-F1 to 0.407, but that model cannot be applied to English questions and is reported only for reference. The most informative features were the digit edit distance between the answers, swapped answer blanks, the relative error and question length.

5.5 Knowledge Tracing

On ASSISTments 2009 with a learner-disjoint 80/20 split (74,867 test answers), BKT fitted per skill by expectation–maximisation reached an AUC of 0.771

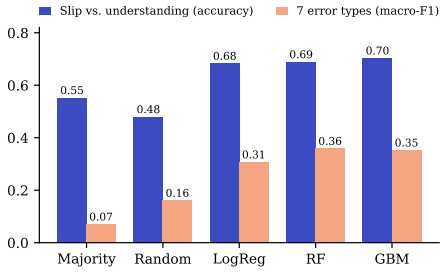


Fig. 5. Real-mistake classifiers against baselines (ScratchMath).

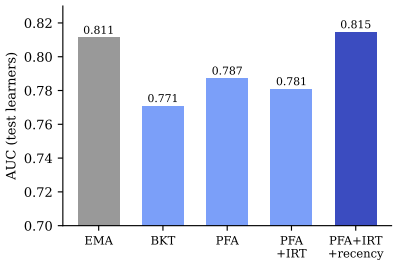


Fig. 6. Next-answer prediction on ASSISTments 2009 (test learners).

and PFA 0.787 (Fig. 6). A simple exponential moving average of recent answers reached 0.811, reflecting a known property of this dataset: skill-builder problem sets end after a run of correct answers, so recent performance is highly predictive. Adding the recency features and Rasch item difficulty to PFA gave the highest AUC, 0.815 (95% CI over learners 0.794–0.837), with the lowest root-mean-square error (0.396) and log-loss (0.476); the Rasch regularisation strength was chosen on validation learners. Its advantage over the moving average is not statistically significant (paired learner bootstrap: +0.005, 95% CI −0.017 to +0.024), but it is clearly better than BKT, whose interval (0.744–0.802) lies almost entirely below it. StepWise uses BKT for the mastery display because its learned-state probability is directly interpretable by teachers. BKT parameters fitted on the matching ASSISTments skills are used for seven of StepWise's eight skills. In a simulation with 40 practice items per learner, choosing the least-mastered skill and items near 70% predicted success left 74% of skills learned, compared with 64% for random or round-robin selection. On a simulated class with four planted groups, K-Means recovered the groups with an adjusted Rand index of 0.82.

5.6 Ablation Studies

Table 6 removes feature groups from the two classifiers trained on real data. For symbols, HOG features carry almost all of the information (92.3% accuracy alone, against 75.2% for the zoning densities); zoning and the aspect ratio add little on top. For real mistakes, the answer-pattern features are the most useful single group, and removing them (question and image statistics only) costs 4.7 points of slip accuracy and 0.08 AUC. The scratchwork-image statistics improve the slip AUC (0.722 to 0.749) and the question statistics improve the seven-way accuracy (40.7% to 49.1%).

Table 6. Feature ablations on held-out real data. Left: SVM symbol recognizer on CROHME test symbols. Right: ScratchMath slip-vs-understanding (logistic regression) and seven-type (random forest) classifiers.

Symbol features	Acc.	F1	Mistake features	Slip acc.	Slip AUC	7-type acc.
HOG only	0.923	0.894	Answer only	0.657	0.722	0.407
Zoning only	0.752	0.746	Answer + question	0.677	0.757	0.491
HOG + zoning	0.925	0.898	Answer + image	0.663	0.749	0.424
All (374)	0.924	0.898	Question + image	0.637	0.683	0.378
			All (42)	**0.683**	**0.765**	**0.491**

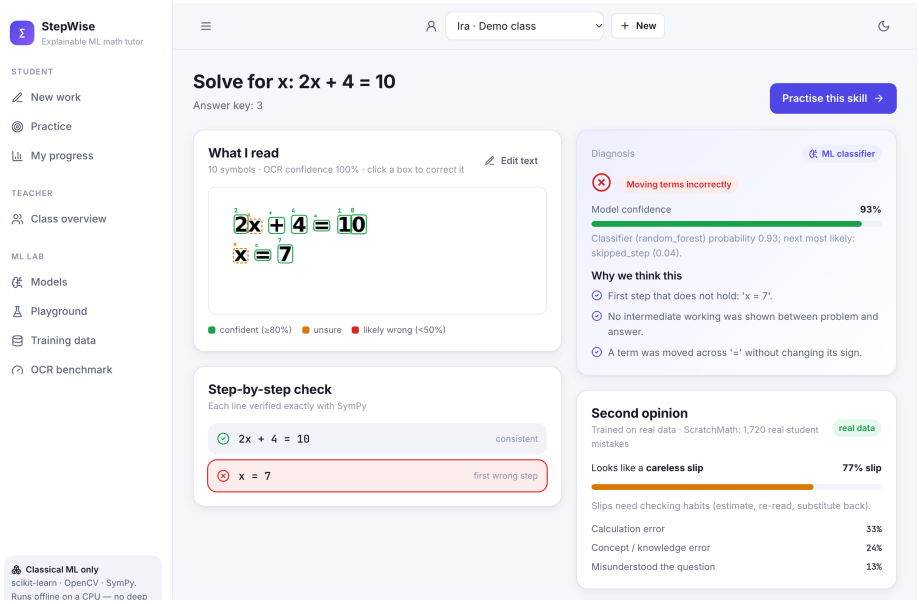


Fig. 7. Analysis screen: recognised symbols with confidence colours, step-by-step verification, the evidence-based diagnosis and the real-data second opinion.

5.7 System Characteristics

Fig. 7 shows the student-facing analysis screen: each recognised symbol is drawn on the photograph and coloured by confidence, the first incorrect step is highlighted, and the diagnosis lists its supporting evidence alongside the real-data second opinion. All trained models together occupy about 25 MB. Processing one photograph took about 0.3 s on a laptop CPU (preprocessing 110 ms, recognition 11 ms, verification 97 ms, diagnosis 82 ms), with no network access. The backend is covered by 175 automated tests, and retraining every model from scratch with a fixed seed reproduces all reported numbers exactly.

5.8 Limitations

CROHME images are rendered from tablet handwriting rather than photographed paper; the degradation study approximates phone photographs, but real photographs of worksheets remain to be measured. Only linear expressions are harvested, and rotation beyond a few degrees still hurts segmentation. Count alignment can mislabel a crop when two segmentation errors cancel out. Segmentation assumes that symbols do not touch, so joined digits are not separated. The fine-grained misconception classifier is trained on generated data, and ScratchMath consists of Chinese word problems with a coarser label set. The adaptive-practice evaluation uses simulated learners.

6 Conclusion

We presented StepWise, a tutoring system that reads handwritten mathematics, proves which step is wrong, explains the likely misconception and adapts practice, using only classical machine learning and exact symbolic computation. Training on real handwriting from CROHME and HASYv2 raised symbol accuracy on unseen writers from 55.4% to 92.4%, showing that synthetic font data alone is not a substitute for real writers. On the same real test expressions the classical recognizer read twice as many expressions exactly as a zero-shot transformer OCR model while being 28 times faster and 160 times smaller, and illumination normalisation made it robust to uneven lighting. On real student data, a classifier trained on 1,720 real mistakes separated careless slips from understanding gaps significantly better than chance (68.3%), and knowledge tracing predicted next answers with an AUC of 0.815. The whole system is about 25 MB and runs offline on a CPU, which makes it suitable for classrooms without GPUs or reliable internet. Future work will collect and label photographs of English classroom worksheets to evaluate the fine-grained diagnosis on real data, fine-tune both recognizers on such photographs, and run a classroom pilot to measure learning gains.

References

1. Brown JS, Burton RR (1978) Diagnostic models for procedural bugs in basic mathematical skills. Cognitive Science 2(2):155–192
2. Koedinger KR, Anderson JR, Hadley WH, Mark MA (1997) Intelligent tutoring goes to school in the big city. International Journal of Artificial Intelligence in Education 8:30–43
3. Song D, Xu T, Zhang YF, Li H, Yan Z, Fan X, Li H, Sun L, Wen Q (2026) Can MLLMs read students' minds? Unpacking multimodal error analysis in handwritten math. In: Proceedings of the 27th International Conference on Artificial Intelligence in Education (AIED)
4. VanLehn K (1990) Mind bugs: the origins of procedural misconceptions. MIT Press, Cambridge, MA

5. Mouchère H, Viard-Gaudin C, Zanibbi R, Garain U (2014) ICFHR 2014 competition on recognition of on-line handwritten mathematical expressions (CROHME 2014). In: 14th International Conference on Frontiers in Handwriting Recognition, pp 791–796
6. Mahdavi M, Zanibbi R, Mouchère H, Viard-Gaudin C, Garain U (2019) ICDAR 2019 CROHME + TFD: competition on recognition of handwritten mathematical expressions and typeset formula detection. In: International Conference on Document Analysis and Recognition, pp 1533–1538
7. Zhang J, Du J, Zhang S, Liu D, Hu Y, Hu J, Wei S, Dai L (2017) Watch, attend and parse: an end-to-end neural network based approach to handwritten mathematical expression recognition. Pattern Recognition 71:196–206
8. Li M, Lv T, Chen J, Cui L, Lu Y, Florencio D, Zhang C, Li Z, Wei F (2023) TrOCR: transformer-based optical character recognition with pre-trained models. In: Proceedings of the AAAI Conference on Artificial Intelligence 37(11):13094–13102
9. Dalal N, Triggs B (2005) Histograms of oriented gradients for human detection. In: IEEE Conference on Computer Vision and Pattern Recognition, pp 886–893
10. Cortes C, Vapnik V (1995) Support-vector networks. Machine Learning 20(3):273–297
11. Maji S, Malik J (2009) Fast and accurate digit classification. Technical Report UCB/EECS-2009-159, University of California, Berkeley
12. Thoma M (2017) The HASYv2 dataset. arXiv:1701.08380
13. Corbett AT, Anderson JR (1994) Knowledge tracing: modeling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction 4(4):253–278
14. Pavlik PI, Cen H, Koedinger KR (2009) Performance factors analysis – a new alternative to knowledge tracing. In: Artificial Intelligence in Education, pp 531–538
15. Piech C, Bassen J, Huang J, Ganguli S, Sahami M, Guibas LJ, Sohl-Dickstein J (2015) Deep knowledge tracing. In: Advances in Neural Information Processing Systems 28, pp 505–513
16. Xiong X, Zhao S, Van Inwegen EG, Beck JE (2016) Going deeper with deep knowledge tracing. In: Proceedings of the 9th International Conference on Educational Data Mining, pp 545–550
17. Rasch G (1960) Probabilistic models for some intelligence and attainment tests. Danish Institute for Educational Research, Copenhagen
18. Feng M, Heffernan N, Koedinger K (2009) Addressing the assessment challenge with an online system that tutors as it assesses. User Modeling and User-Adapted Interaction 19(3):243–266
19. Meurer A, Smith CP, Paprocki M et al (2017) SymPy: symbolic computing in Python. PeerJ Computer Science 3:e103
20. Breiman L (2001) Random forests. Machine Learning 45(1):5–32
21. Pedregosa F, Varoquaux G, Gramfort A et al (2011) Scikit-learn: machine learning in Python. Journal of Machine Learning Research 12:2825–2830